HybridAttentionNet: Lightweight Dual-Branch Spatial-Temporal Architecture with Branch Collapse Prevention for Healthcare IoT Malware Detection

Archa Nair U G, Devi Rajeev, Vysakh Kani Kolil
Center for Cybersecurity Systems and Networks
Amrita Vishwa Vidyapeetham
Amritapuri, India
archanairug@gmail.com, devir@am.amrita.edu, vysakh@am.amrita.edu

***Abstract*—Protecting a hospital network turns out to be tougher than the usual IoT benchmarks let on. Infusion pumps and patient monitors run on tight memory and latency budgets, and in clinical settings a prediction usually has to be explainable, not just accurate. While building hybrid spatial-temporal intrusion detectors, the same problem kept surfacing: one encoder soaked up almost all of the attention weight while the other quietly stopped learning. On paper the model had two branches; in practice it did not. We refer to this as *branch collapse*, and fixing it is what most of this paper is about. HybridAttentionNet combines a Bidirectional LSTM temporal encoder with a Residual MLP spatial encoder and fuses them through an entropy-regularized gate that pushes back against lopsided branch usage during training. On the CTU IoT Network Traffic dataset (644,706 flow records, four classes), the model reaches a weighted-F1 of 0.969 and 96.13% test accuracy, with a 5-fold macro-F1 of 0.926 $\pm$ 0.012, indicating stable results. It is also lightweight,with only 998,026 parameters and 2.21 ms per-window inference,so it runs comfortably on constrained edge hardware. Removing the spatial branch drops the macro-F1 by 0.551, confirming that both branches do genuine work.Finally, the SHAP attributions match known HIoT attack signatures, so the model can clearly show why a particular alert was raised.**

***Index Terms*—healthcare IoT security, malware detection, branch collapse prevention, BiLSTM, attention mechanism, explainable AI, SHAP, intrusion detection, deep learning**

I. Introduction

A hospital's attack surface is much bigger than only servers and workstations.Devices like infusion pumps,wearable monitors, and diagnostic machines were mostly made for clinical reliability,not for handling attackers (1). So when ransomware enters the network,it does not just leak the data,it can directly affect the patient care also. And missing a command-and-control (C&C) beacon is very risky, because it gives the malware enough time to spread through the systems before anyone even notices. Conventional IT security tools were simply never built with these constraints in mind.

Hybrid spatial-temporal classifiers suit intrusion detection well, because network traffic carries structure on two levels: inside a single flow and across a sequence of them. While developing these models, we kept hitting the same failure mode, which we call *branch collapse*. Formally, it shows up when Balance$(\boldsymbol{\alpha}) = 1 - |\alpha_1 - \alpha_2|$ drops toward zero, with α_1 and α_2 being the softmax attention weights over the two branches. In that state one encoder takes in almost all of the gradient signal while the other slowly learns nothing at all. Once that happens, it is hard to defend any claim about what the dual-branch model is actually capturing.

We hit this problem directly. A ResNet18+BiLSTM hybrid we built achieved 99.97% accuracy, but the BiLSTM received attention weight 0.01 across every class,the temporal encoder was essentially decorative. HybridAttentionNet addresses this through three mechanisms: entropy regularization on the attention distribution, a scalar gate penalized for straying from the balanced midpoint, and an auxiliary supervision head on the temporal branch to prevent gradient starvation early in training.

The contributions of this paper are:

1) A three-stage architecture evolution study that documents and quantifies branch collapse in hybrid HIoT intrusion detection.
2) HybridAttentionNet: a dual-branch design with entropy-regularized gated fusion, validated through ablation (spatial branch removal reduces macro-F1 by 0.551) and per-class attention analysis.
3) A training pipeline combining focal loss, MixUp, and auxiliary temporal supervision, yielding weighted-F1 of 0.969 and 5-fold macro-F1 of 0.926 ± 0.012.
4) Comparison against six baselines with SHAP analysis linking spatial encoder decisions to recognized HIoT attack signatures.
5) A 998K-parameter model with 2.21 ms inference latency-roughly 11$\times$smaller than the ResNet18+BiLSTM baseline.

II. Related Work

Classical and ML-based IDS. A large portion of the IDS literature still relies on SVMs, Random Forests, and k-NN built on handcrafted per-flow features (2; 3). These approaches treat each flow record independently, so patterns that only emerge across a sequence stay invisible to them. Healthcare-specific variants for IoMT (**? ?**) share this limitation. Class imbalance gets handled through SMOTE (4), focal loss (5), or MixUp (6); we ended up using all three.

Deep and hybrid architectures. LSTMs and GRUs handle temporal flow analysis more naturally (7; 8). Kitsune (9) showed that lightweight autoencoders can catch IoT anomalies without needing deep networks. CNN-based approaches (10) convert flow windows into image-like tensors, and pairing CNNs with LSTMs (11**?**) adds temporal modeling on top. Transformers have also appeared in this space (12), though their parameter counts are hard to justify on constrained edge hardware. SHAP (13) is the standard tool for post-hoc explainability here, but branch-level attribution in lightweight HIoT models has gotten little attention.

Branch collapse. Dual-stream architectures with learned fusion have been studied in video understanding and multimodal learning, but the specific failure of branch collapse in IDS has not been formally characterized. Work like DBAD runs two encoders for time-series anomaly detection without tracking whether both branches are actually doing anything. To our knowledge, this paper is the first to define a quantitative collapse metric and treat branch utilization as a primary experimental observable in HIoT intrusion detection.

III. Methodology

A. Dataset

All experiments were done on the CTU IoT Network Traffic dataset (14), which has 644,706 flow records spread across twelve CSV files. We kept only four useful classes: Benign (144,043), DDoS (135,443), PortScan (99,883), and C&C (6,821). The imbalance, where C&C is only about 1%, is kept on purpose, because real hospital traffic also has very rare beaconing compared to normal activity.

B. Data Preprocessing and Feature Engineering

We pulled ten raw per-flow features-*duration, orig_bytes, resp_bytes, missed_bytes, orig_pkts, orig_ip_bytes, resp_pkts, resp_ip_bytes, conn_state, proto*-and then built seven behavioral ratios on top of them: payload asymmetry, byte ratio, packet ratio, bytes per packet in each direction, bytes per second, and connection efficiency. After clipping outliers at the 0.5th and 99.5th percentiles, we were left with 17 features per flow. Those extra ratios were not optional in practice-in early runs, raw byte counts on their own could not cleanly tell DDoS apart from large but benign bulk transfers.

We grouped flows by source IP, sorted them by timestamp, and cut them into non-overlapping 16-flow windows, which gave 23,976 windows before any balancing. We picked the window length to be long enough to hold a short attack sequence yet short enough to process in real time at the edge. Each window feeds two inputs that are kept deliberately separate: a (16×17) sequence tensor for the temporal branch, and a 10-dimensional aggregate vector for the spatial branch, covering IP diversity, port entropy, inter-flow delta statistics, state transition counts, protocol fractions, and mean bytes per second. Keeping these inputs from overlapping nudges each encoder to specialize instead of relearning the same features twice.

We split the data along a per-class temporal boundary (70% train, 15% validation, 15% test), so chronological order stays intact within each class.We kept timestamps out of the model inputs so that it does not take easy temporal shortcuts. To handle the imbalance in the training set, RandomUnderSampler brought the majority classes down to 5,000 windows, while SMOTE raised C&C from 296 up to 5,000.On top of this, we also applied MixUp augmentation during training as an extra regularizer.

C. Proposed Architecture: HybridAttentionNet

The design went through three iterations. The first one, a standalone BiLSTM (Baseline 1), could model flow sequences but had no way to bring in the aggregate structural statistics. Next, we added a ResNet18 spatial branch that worked on 32×32 window images (Baseline 2), and this pushed accuracy to 99.97%. But the attention weights told a different story,ResNet18 was holding 0.99 while BiLSTM stayed at 0.01 uniformly across all classes, which is a full case of branch collapse. With 11.47M parameters and image-conversion overhead, it was also impractical for hospital edge hardware. HybridAttentionNet replaces ResNet18 with a Residual MLP on the raw 10-dimensional spatial vector.

Temporal Branch. The temporal branch processes (16×17) sequences through a two-layer BiLSTM with 128 hidden units per direction, followed by self-attention pooling to produce $\mathbf{t} \in \mathbb{R}^{192}$. Bidirectionality is important here because some anomalies only make sense in context-a teardown pattern that looks ordinary in isolation can look suspicious once you know what preceded it. An auxiliary classification head with $\lambda_{\text{aux}} = 0.3$ provides direct supervision to this branch; without it, the gate consistently suppressed temporal gradients before the BiLSTM had a chance to learn anything useful. **Spatial Branch.** So here we take the 10-feature summary vector and first expand it to 256 dimensions,normalize it a bit,pass it through GELU, and then send it through two Residual MLP blocks, which finally gives us $\mathbf{s} \in \mathbb{R}^{192}$.We keep a dropout of 0.4 in both branches so the model does not overfit.One thing we noticed is that those residual connections really helped, because the moment we removed them, this branch kept doing badly on the C&C class.We feel it is because the beaconing patterns are a bit subtle, so the network needs some depth before it can actually catch them.

Entropy-Regularized Gated Fusion. Now to combine the two branches,we calculate the attention weights as $\boldsymbol{\alpha} = \text{softmax}([\|\mathbf{t}\|_2, \|\mathbf{s}\|_2])$.The main problem we faced was that one branch would take over and the other would just stop learning.So to stop this collapsing, we add a small penalty term $-\lambda_H H(\boldsymbol{\alpha})$ to the loss, where $H(\boldsymbol{\alpha}) = -\sum_i \alpha_i \log(\alpha_i + \varepsilon)$.There is also a simple gate $g = \sigma(w)$, started at 0.5,which has its own small balance penalty.So putting everything together,the full loss is:

$$L = L_{\text{focal}} + 0.3 L_{\text{CE}}^{\text{aux}} - 0.05 H(\boldsymbol{\alpha}) + 0.01 \|g - 0.5\|^2 \quad (1)$$

After this, the combined output $\mathbf{c} = [g\mathbf{t};\ (1-g)\mathbf{s}] \in \mathbb{R}^{384}$ goes into a simple $384 \to 128 \to 4$ classifier.After trying different

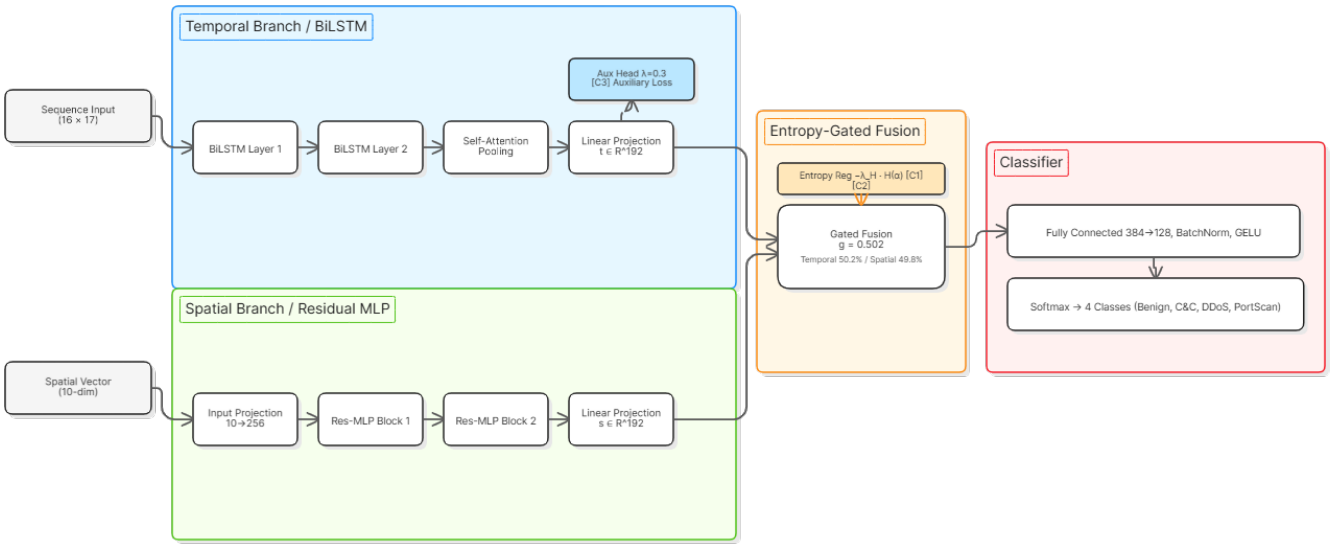


Fig. 1. HybridAttentionNet. Temporal branch (blue): two-layer BiLSTM with self-attention pooling producing $\mathbf{t} \in \mathbb{R}^{192}$. Spatial branch (green): Residual MLP on 10-dim aggregate features producing $\mathbf{s} \in \mathbb{R}^{192}$. Entropy-regularized gated fusion (orange) prevents collapse. Auxiliary head (dashed) prevents BiLSTM gradient starvation. Gate: $g = 0.502$.

values, we finally settled on $\lambda_H = 0.05$.This was the smallest value that still kept the balance above 0.4 for every class; if we went any lower,the collapsing problem slowly came back for DDoS and PortScan.

Training Setup. For training we used AdamW (**?**) instead of SGD, because our loss in (1) is a bit complex and works better with AdamW.When we tried SGD with momentum, the early training was quite shaky. The rest of the settings were simple: learning rate 3×10^{-4}, weight decay 10^{-3}, batch size 512, OneCycleLR cosine annealing, up to 80 epochs with patience-12 early stopping on validation macro-F1, and gradient clipping at 1.0. We ran everything over five seeds (42, 7, 123, 999, 2024) to make sure the results are stable. In the end, the trained model is only about 4 MB on disk and needs roughly 12 M FLOPs per window,which is quite light. We built the whole thing using PyTorch 2.1, scikit-learn 1.3,and shap 0.43.

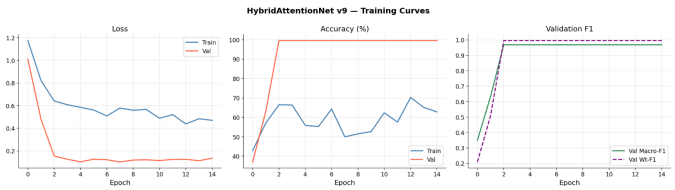


Fig. 2. Training curves across 15 epochs. The validation loss reaches ≈0.10 by epoch 3, and both F1 metrics settle at 0.969 shortly after. The gap between the training and validation curves stays narrow throughout.

IV. Experimental Results

A. Architecture Comparison

The ResNet18+BiLSTM numbers look great, but the branch weights tell the real story.ResNet18 sits at 0.99 and BiLSTM at just 0.01, giving Balance(α) = 0.020, which means it is almost fully collapsed. Our model loses a few accuracy points (96.13%), but in return both branches are actually used, with Balance between 0.460 and 0.775 across classes.We use the 5-fold macro-F1 of 0.926 ± 0.012 as our main number instead of accuracy, and this is on purpose.C&C is only 1.8% of the test set, so even if the model gets all C&C wrong, the accuracy barely drops, less than 0.03%.So a model that ignores C&C can still show a high 99.97%, but macro-F1 treats every class equally and cannot be fooled like that.

TABLE I
Architecture Comparison

Metric	BiLSTM	ResNet18	Proposed
Parameters	∼300K	11.47M	**998K**
Test Accuracy	∼94%	99.97%	96.13%
Weighted-F1	∼0.93	∼1.00	**0.969**
5-Fold Macro-F1	-	-	**0.926 ± 0.012**
BiLSTM attn.	1.00	0.01 (collapsed)	0.23–0.61
Image conversion	No	Yes	**No**
Edge deployable	Yes	No	**Yes**
Latency / Throughput	-	-	**2.21 ms / 814K f/s**

B. Per-Class Performance

DDoS and PortScan come through with no errors at all. C&C is harder: a precision of 0.28 reflects benign flows being pulled into the C&C bucket, while recall of 0.72 means roughly one in four genuine C&C flows slips through undetected. With only 64 C&C test windows sourced from a single IP address, the 95% CI of [0.326, 0.482] is wide enough that any single-split C&C F1 estimate should be treated carefully.

TABLE II
PER-CLASS TEST PERFORMANCE (3,614 WINDOWS)

Class	P	R	F1	95% CI
Benign	0.99	0.91	0.95	[0.938, 0.955]
Malicious C&C	0.28	0.72	0.41	[0.326, 0.482]
Malicious DDoS	1.00	1.00	1.00	[0.997, 1.000]
Malicious PortScan	1.00	1.00	1.00	[0.996, 1.000]
Macro avg	0.82	0.91	0.84	-
Wtd. avg	0.98	0.96	0.97	-

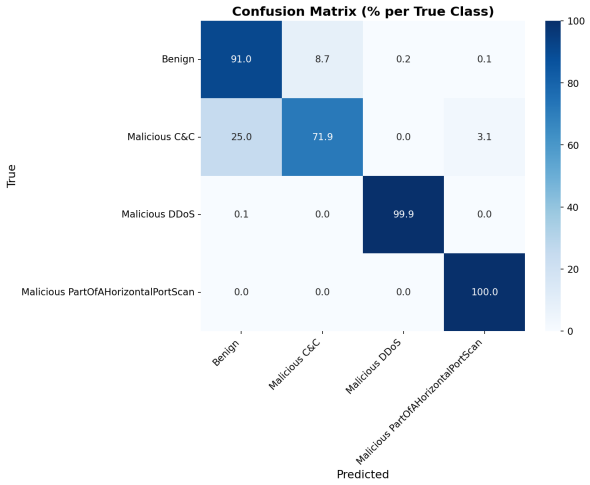


Fig. 3. Normalized confusion matrix (3,614 test windows). DDoS and PortScan: perfect classification. C&C recall is 72%; the remaining 28% land in Benign, attributable to behavioral overlap and thin test support.

TABLE III
ROBUSTNESS ACROSS SEEDS AND FOLDS

Method	Accuracy	Macro-F1	Wtd-F1
Single run	0.9613	0.8372	0.9686
5-seed mean $\pm$ std	0.955 ± 0.023	0.849 ± 0.046	0.961 ± 0.020
5-seed 95% CI	[0.923, 0.986]	[0.785, 0.913]	[0.933, 0.989]
5-Fold CV	0.992 ± 0.001	$\mathbf{0.926 \pm 0.012}$	-

C. Robustness

When we ran five different seeds (42,7,123,999,2024), the macro-F1 values came between 0.792 and 0.932. Almost all of this spread is because of C&C, which changes a lot depending on which 64 windows fall into the test set. No seed gave a very big outlier,so we feel this is just sampling noise and not any training instability. That is why we trust the 5-fold macro-F1 of 0.926 ± 0.012 the most, since it is measured on naturally distributed and unaugmented data.

D. Ablation

The spatial-only row is the one that matters most: dropping from 0.837 to 0.286 puts to rest any suggestion that the dual-branch structure is cosmetic. MixUp removal costs 0.019-small but consistent across runs.

TABLE IV
ABLATION STUDY - MACRO-F1

Variant	Macro-F1	Δ
Full model	0.837	-
No MixUp	0.818	-0.019
Temporal only	0.805	-0.032
Spatial only	0.286	-0.551
No aux loss	0.943	$+0.106$
No entropy reg.	0.932	$+0.094$

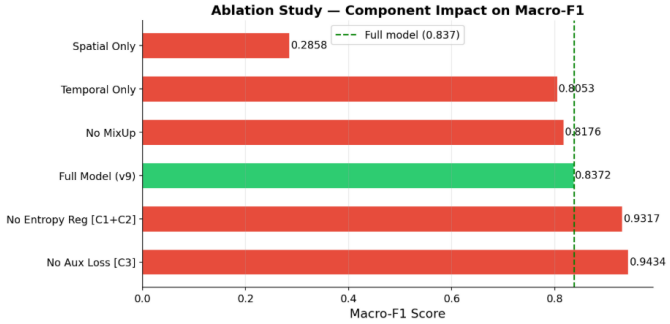


Fig. 4. Ablation macro-F1. Removing the spatial branch causes the biggest drop ($\Delta = -0.551$). The entropy-regularization and aux-loss rows actually score higher once removed-that is the cost of interpretability, not a design mistake.

The entropy regularization and auxiliary loss rows go in the opposite direction, and that result deserves a direct explanation rather than being glossed over. Without entropy regularization, DDoS and PortScan drift to temporal attention weights of 0.995 and 0.998 respectively (see Table VI), leaving the spatial encoder contributing roughly 0.5% of the fusion signal for those classes. At that point the model has become a single-branch temporal classifier with a dormant spatial branch that happens to still be in the graph. With entropy regularization active, the C&C spatial weight holds at 0.770-the region where the spatial encoder provides the most value. The 0.094-0.106 macro-F1 cost is the price of having an architecture that actually behaves as claimed, and it should be read alongside Table VI rather than in isolation.

E. Baselines

TABLE V
BASELINE COMPARISON (SAME SPLIT). SEQ.: SEQUENTIAL FLOW MODELING WITHIN A WINDOW.

Model	Acc.	MF1	WF1	Seq.
XGBoost	0.998	0.991	0.998	No
Random Forest	0.998	0.987	0.998	No
Transformer	0.994	0.953	0.993	Yes
Proposed	0.961	0.837	0.969	Yes
CNN	0.947	0.827	0.959	No
CNN-LSTM	0.932	0.808	0.950	Yes
LSTM	0.934	0.795	0.951	Yes

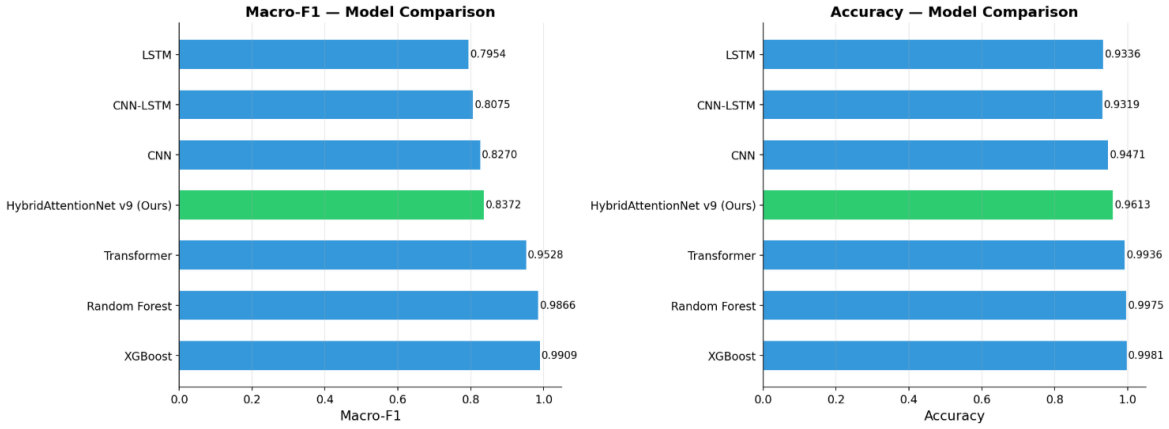


Fig. 5. Macro-F1 and accuracy across all evaluated models. XGBoost and Random Forest lead on macro-F1. Among neural sequence models, the proposed model ranks above CNN, CNN-LSTM, and LSTM.

XGBoost and Random Forest come out on top for macro-F1, which is not surprising given how well-suited both are to structured tabular features. The practical problem is memory: a 300-tree forest typically needs several hundred megabytes, which rules it out on microcontroller-class edge hardware. Neither method models flow ordering either, which matters when the attack pattern only becomes visible across a sequence.

The Transformer reaches macro-F1 of 0.953, which is 0.116 above the proposed model-and we are not going to downplay that gap. Without a dual-branch constraint, the Transformer can optimize purely for classification accuracy, and on this dataset it does exactly that. The more relevant question for a clinical deployment is whether a clinician or auditor can trace what drove a specific alert. HybridAttentionNet answers that per class through Table VI; the Transformer does not. Whether the 0.116 gap is acceptable depends on the deployment context. Among neural baselines that model temporal flow ordering, the proposed model ranks highest on all three reported metrics.

F. Branch Attention Analysis

TABLE VI
PER-CLASS BRANCH ATTENTION WEIGHTS. BAL. $= 1 - |\alpha_{\text{TEMP}} - \alpha_{\text{SPAT}}|$; NEAR 1.0 INDICATES BALANCE, NEAR 0 INDICATES COLLAPSE.

Class	Temp.	Spat.	g	Bal.
Benign	0.612	0.388	0.502	0.775
Malicious C&C	0.230	0.770	0.502	0.460
Malicious DDoS	0.995	0.005	0.502	0.011
Malicious PortScan	0.998	0.002	0.502	0.005
ResNet18+BiLSTM	0.010	0.990	-	0.020

The global gate at $g = 0.502$ paints a balanced picture; the per-class numbers are far more informative. C&C is strongly spatial-dominant at 0.770. Beaconing to a fixed set of endpoints at regular intervals produces low port entropy and predictable connection-state sequences-exactly the kind of aggregate statistics the spatial branch is built to catch. On their own, the individual flow sequences look unremarkable. DDoS and PortScan are the reverse case: flooding and systematic probing both leave clear sequential signatures that the temporal encoder reads easily, so the spatial aggregates add little there. Benign traffic, meanwhile, leans on both branches in a meaningful way. A single-branch model has no way to shift like this. The per-class spread in Table VI is, we would argue, exactly the behavior the dual-branch design was meant to produce.

V. EXPLAINABILITY

We ran SHAP on the spatial branch just to check one thing,whether the model is really looking at proper attack behaviour,or just picking up some quirks of the CTU data.This matters a lot once the model is used in front of clinical staff or in an audit.

For C&C,the top features are `state_transitions` (0.120) and `port_entropy` (0.092),which makes sense, since beaconing keeps contacting the same endpoints again and again,so the ports stay limited and the pattern keeps repeating.For DDoS,it mostly uses `state_transitions` (0.074) and `bps_mean` (0.066),because the many rejected connections and constant flooding push these values up.For PortScan,it looks at `bps_mean` (0.033) and `state_transitions` (0.020), which fits the probing of many destinations.

So overall,these are more or less the same things an experienced analyst would also expect.But this alone does not fully prove the model learned something general,so we would still need to test it on other datasets before saying more.

VI. DISCUSSION

The 99.97% accuracy of the ResNet18+BiLSTM baseline is misleading. Balance$(\boldsymbol{\alpha}) = 0.020$ for that model; the BiLSTM carries 0.01 weight across all classes, yet the accuracy number gives no indication that anything is wrong. HybridAttentionNet accepts lower raw accuracy in return for Balance values of 0.460 to 0.775—a result that is directly verifiable from the attention weights, not just asserted.

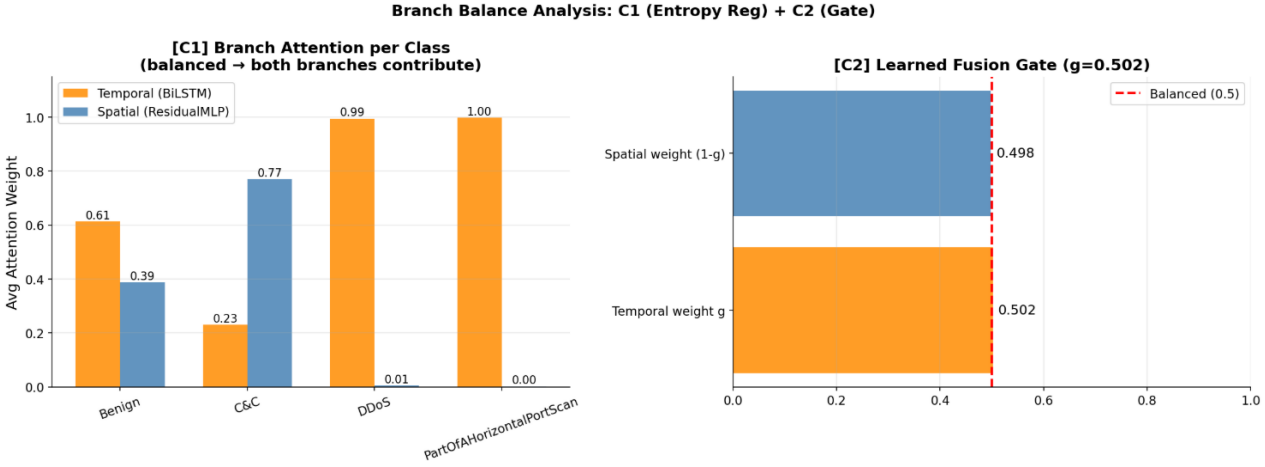


Fig. 6. Per-class branch attention (left) and learned gate $g = 0.502$ (right). C&C is spatial-dominant (0.770); DDoS and PortScan are temporal-dominant (0.995, 0.998). The ResNet18+BiLSTM baseline holds at 0.010 temporal across every class.

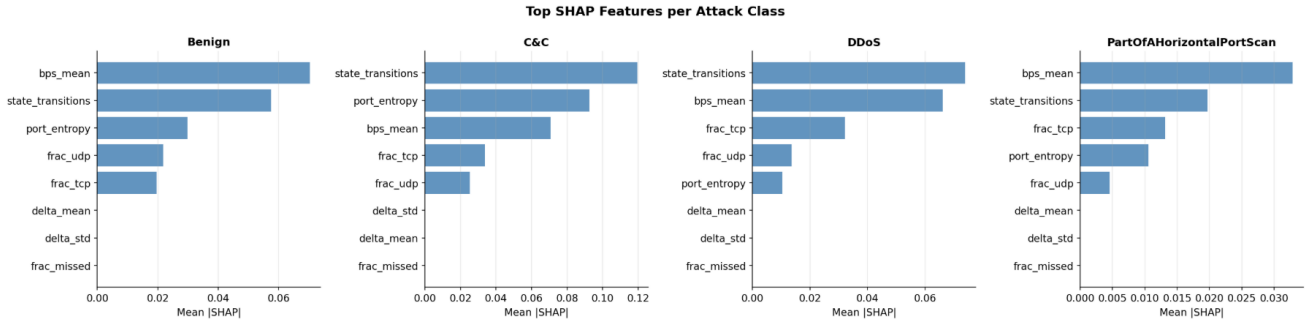


Fig. 7. SHAP GradientExplainer applied to the spatial branch. C&C: `state_transitions` and `port_entropy`. DDoS: `state_transitions` and `bps_mean`. PortScan: `bps_mean` and `state_transitions`. Each one lines up with established HIoT attack signatures.

We prefer the 5-fold macro-F1 of 0.926 ± 0.012 over the single-split 0.837 because the C&C partition is fragile: 64 test windows from a single source IP, with a 95% CI of [0.326, 0.482]. XGBoost, Random Forest, and the Transformer all outperform the proposed model on macro-F1, and we report this without qualification. The case for HybridAttentionNet rests on verified dual-branch utilization, within-window sequential modeling, 4 MB storage, and 2.21 ms per-window inference—properties that matter considerably more than benchmark rank when the target deployment is constrained hospital edge hardware.

The ablation rows for entropy regularization and auxiliary loss need a simple, honest comment here, that is, removing either one of them actually pushes the macro-F1 slightly up. But this is not a design flaw.Without entropy regularization, DDoS and PortScan drift to temporal attention weights of 0.995 and 0.998, leaving the spatial encoder contributing roughly 0.5% of the fusion signal. The architecture degenerates into a single-branch classifier with a dormant spatial branch. The 0.094-0.106 macro-F1 cost is the price of maintaining the dual-branch behavior that makes the model interpretable and auditable—and it should always be read alongside Table VI, not in isolation.

The most significant limitation is the single-dataset evaluation. Whether results transfer to Bot-IoT, TON_IoT, CICIDS2017, or real hospital traffic is an open question. We also did not profile latency on ARM-class hardware, which is what actual edge deployment would require, and paired significance tests across baselines were not conducted. These remain the primary agenda for follow-up work.

VII. Conclusion

HybridAttentionNet achieves 50.2%/49.8% balanced branch utilization, compared to the 0.01/0.99 collapse in the ResNet18+BiLSTM baseline. Removing the spatial branch costs 0.551 in macro-F1, confirming that both encoders are genuinely contributing. The full model reaches 0.837 on the single test split and 0.926 ± 0.012 across five folds. Entropy regularization and auxiliary supervision reduce raw macro-F1 by 0.094–0.106, but they preserve the dual-branch behavior—particularly the C&C spatial weight of 0.770—that makes the architecture interpretable rather than merely accurate.At just 998K parameters and 2.21 ms per window, the model sits well within the reach of hospital edge hardware. The SHAP attributions give feature importances that agree with the known HIoT attack signatures, and this offers a good foothold for

explainability in regulated clinical settings, where a prediction has to be properly explained and not simply trusted.

REFERENCES

[1] K. Kandasamy, S. Srinivas, K. Achuthan, and V. P. Rangan, "Digital healthcare-cyberattacks in asian organizations: an analysis of vulnerabilities, risks, nist perspectives, and recommendations," *IEEE access*, vol. 10, pp. 12 345–12 364, 2022.

[2] W. Lee and S. J. Stolfo, "A framework for constructing features and models for intrusion detection systems," *ACM transactions on Information and system security (TiSSEC)*, vol. 3, no. 4, pp. 227–261, 2000.

[3] R. Sommer and V. Paxson, "Outside the closed world: On using machine learning for network intrusion detection," in *2010 IEEE symposium on security and privacy*. IEEE, 2010, pp. 305–316.

[4] N. V. Chawla, K. W. Bowyer, L. O. Hall, and W. P. Kegelmeyer, "Smote: synthetic minority over-sampling technique," *Journal of artificial intelligence research*, vol. 16, pp. 321–357, 2002.

[5] T.-Y. Lin, P. Goyal, R. Girshick, K. He, and P. Dollár, "Focal loss for dense object detection," in *Proceedings of the IEEE international conference on computer vision*, 2017, pp. 2980–2988.

[6] H. Zhang, M. Cisse, Y. N. Dauphin, and D. Lopez-Paz, "mixup: Beyond empirical risk minimization," *arXiv preprint arXiv:1710.09412*, 2017.

[7] S. A. Althubiti, E. M. Jones, and K. Roy, "Lstm for anomaly-based network intrusion detection," in *2018 28th International telecommunication networks and applications conference (ITNAC)*. IEEE, 2018, pp. 1–3.

[8] S. Hochreiter and J. Schmidhuber, "Long short-term memory," *Neural computation*, vol. 9, no. 8, pp. 1735–1780, 1997.

[9] Y. Mirsky, T. Doitshman, Y. Elovici, and A. Shabtai, "Kitsune: an ensemble of autoencoders for online network intrusion detection," *arXiv preprint arXiv:1802.09089*, 2018.

[10] M. A. Ferrag, L. Maglaras, S. Moschoyiannis, and H. Janicke, "Deep learning for cyber security intrusion detection: Approaches, datasets, and comparative study," *Journal of Information Security and Applications*, vol. 50, p. 102419, 2020.

[11] A. Aldweesh, A. Derhab, and A. Z. Emam, "Deep learning approaches for anomaly-based intrusion detection systems: A survey, taxonomy, and open issues," *Knowledge-Based Systems*, vol. 189, p. 105124, 2020.

[12] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, "Attention is all you need," *Advances in neural information processing systems*, vol. 30, 2017.

[13] S. M. Lundberg and S.-I. Lee, "A unified approach to interpreting model predictions," *Advances in neural information processing systems*, vol. 30, 2017.

[14] S. Garcia, M. Grill, J. Stiborek, and A. Zunino, "An empirical comparison of botnet detection methods," *computers & security*, vol. 45, pp. 100–123, 2014.